

Gameverse vizsgaremek dokumentáció

Szoftverfejlesztő és -tesztelő

Projekt neve: Gameverse

Készítő: Molnár Péter

Osztály: 13.B

Dátum: 2026.04.28.

Projekt típusa: full-stack webalkalmazás

Tartalomjegyzék

Tartalomjegyzék.....	2
1. Bevezetés.....	4
2. Projekt specifikáció.....	4
2.1 Projekt célja.....	4
2.2 Célközönség.....	5
2.3 Projekt hatóköre.....	5
2.4 Mérhető célok.....	5
3. Funkcionális és nem funkcionális követelmények.....	5
3.1 Funkcionális követelmények.....	5
3.2 Nem funkcionális követelmények.....	6
4. Technológiák és fejlesztőeszközök.....	7
4.1 Backend.....	7
4.2 Frontend.....	7
4.3 Adatbázis.....	7
4.4 Fejlesztőeszközök.....	8
5. Fejlesztési módszertan és csapatmunka.....	8
5.1 Feladatkövetés.....	8
5.2 Verziókezelés.....	8
5.3 Szerepek.....	9
5.4 Minőségbiztosítási terv.....	9
6. Rendszeráttekintés és architektúra.....	9
6.1 Magas szintű architektúra.....	10
6.2 Backend rétegek.....	11
6.3 Frontend szerkezet.....	11
6.4 Jogosultsági szintek.....	11
7. Adatbázis dokumentáció.....	12
7.1 Táblák.....	12
7.2 Kapcsolatok.....	13
7.3 ER diagram.....	13
7.4 Normalizálás.....	13
7.5 Indexek és megszorítások.....	13
8. API dokumentáció.....	14
8.1 Auth API.....	14
8.2 Game API.....	15
8.3 Score és leaderboard API.....	16

8.4 Admin user API.....	17
8.5 Admin game API.....	18
8.6 Hibakezelés.....	19
9. Telepítési és futtatási útmutató.....	19
9.1 Előfeltételek.....	19
9.2 Adatbázis beállítása.....	20
9.3 Backend futtatása.....	20
9.4 Frontend futtatása.....	21
9.5 Admin belépés.....	21
10. Használati útmutató.....	22
10.1 Vendég felhasználó.....	22
10.2 Regisztráció és bejelentkezés.....	22
10.3 Profiloldal.....	23
10.4 Kezdőlap.....	23
10.5 Játékololdal.....	23
10.6 Globális ranglista.....	24
10.7 Admin felület.....	24
11. Tesztelési dokumentáció.....	24
11.1 Tesztelési célok.....	24
11.2 Automatikus backend tesztek.....	25
11.3 Futtatott ellenőrzések.....	26
11.4 Megjegyzés a tesztkörnyezetről.....	27
11.5 Manuális tesztesetek.....	27
12. Kódminőség és biztonsági megoldások.....	29
12.1 Tiszta kód.....	29
12.2 Biztonság.....	29
12.3 Lokalizáció.....	29
13. Továbbfejlesztési lehetőségek.....	29
13.1 Funkcionális bővítések.....	30
13.2 Nem funkcionális bővítések.....	30
14. Vizsgabemutató vázlat.....	30
15. Angol nyelvű bemutató.....	31
16. Források.....	32

1. Bevezetés

A Gameverse egy webes játékportál, amelyben a felhasználók játékokat böngészhetnek, játszhatnak, értékelhetik őket, valamint pontokat gyűjthetnek. A rendszer célja, hogy egy helyen tegye elérhetővé a különböző játékokat, miközben biztosítja a ranglista, a felhasználói profil, a többnyelvű felület és az adminisztrációs kezelés lehetőségét is.

A projekt valós problémára ad megoldást: egy egyszerűen bővíthető játékgyűjteményt biztosít, ahol az adminisztrátor új játékokat adhat hozzá, a felhasználók pedig egy modern, reszponzív felületen használhatják az alkalmazást. A rendszer külön kezeli a beágyazott külső játékokat és a saját inline HTML játékokat. Inline játékok esetén a pontmentés automatikusan működhet, amennyiben a játék a megadott JavaScript bridge-en keresztül pontszámot küld.

A vizsgakövetelmények alapján a projekt teljes full-stack alkalmazás: adatbázisból, backendből és frontendből áll. A backend REST API-t biztosít, a frontend pedig React-alapú kliensalkalmazásként használja ezeket a végpontokat.

2. Projekt specifikáció

2.1 Projekt célja

A projekt célja egy olyan játékportál elkészítése, amely:

- játékokat listáz és részletes játékdalt biztosít,
- lehetővé teszi felhasználók regisztrációját és bejelentkezését,
- profilkezelést biztosít profilkép-feltöltéssel,
- pontszámokat ment és ranglistát jelenít meg,
- támogatja a like/dislike értékeléseket,

- admin felületet biztosít játékok és felhasználók kezelésére,
- magyar és angol nyelvű felületet ad,
- responzív kialakítású, így asztali és mobil eszközön is használható.

2.2 Célközönség

Az alkalmazás célközönsége olyan felhasználókból áll, akik böngészőben szeretnének egyszerű játékokat kipróbálni és más felhasználókkal ranglistán versenyezni.

2.3 Projekt hatóköre

A projekt webalkalmazásként készült el. Tartalmaz publikus oldalakat, felhasználói funkciókat és adminisztrációs funkciókat. A projekt része a Spring Boot backend, a React frontend, a MySQL adatbázis és a teszt kódok.

Nem része a projektnek natív mobilalkalmazás vagy natív asztali alkalmazás, mivel a vizsgakövetelmények szerint responzív webes kliens is elfogadható.

2.4 Mérhető célok

- A felhasználó tudjon regisztrálni és bejelentkezni.
- A játéklista adatbázisból töltődjön be.
- Az admin tudjon játékot létrehozni, módosítani, törölni és publikálási státuszt kezelni.
- A felhasználó tudjon pontszámot menteni és ranglistát nézni.
- A frontend használja a backend REST API-ját.
- A rendszer tartalmazzon legalább 5 adatbázistáblát.
- A projekt fordítható és futtatható legyen lokális fejlesztői környezetben.

3. Funkcionális és nem funkcionális követelmények

3.1 Funkcionális követelmények

1. Regisztráció és bejelentkezés: a felhasználók e-mail címmel, felhasználónévvel és jelszóval regisztrálhatnak, majd JWT token alapú belépéssel használhatják a védett funkciókat.
2. Profilkezelés: a felhasználó módosíthatja nevét, e-mail címét, jelszavát, profilképet tölthet fel és törölheti a saját profilját.
3. Játéklista és keresés: a kezdőlapon megjelennek az aktív játékok, a felhasználó cím vagy leírás alapján kereshet.
4. Játékoldal: a felhasználó megnyithatja a játékot, megtekintheti a leírást, a játékleírást és a játékhoz tartozó ranglistát.
5. Pontmentés: bejelentkezett felhasználó pontszámot menthet. Inline HTML játékoknál a pontmentés automatikusan működhet a beépített bridge segítségével.
6. Ranglista: globális és játékonkénti ranglista érhető el, pontszám szerinti rendezéssel és dátumszűréssel.
7. Értékelés: a bejelentkezett felhasználó like vagy dislike visszajelzést adhat egy játékra.
8. Admin felhasználókezelés: az admin listázhat, létrehozhat, módosíthat és törölhet felhasználókat.
9. Admin játékkezelés: az admin játékot adhat hozzá, képet tölthet fel, szerkeszthet, törölhet, előnézetet nyithat és publikálási státuszt állíthat.
10. Többnyelvűség: a felület magyar és angol nyelven is használható.

3.2 Nem funkcionális követelmények

1. Reszponzív megjelenés: az alkalmazás asztali és mobil eszközökön is használható.
2. Biztonság: a védett műveletek JWT tokennel védettek, admin műveletekhez admin jogosultság szükséges.
3. Karbantarthatóság: a frontend funkciók feature-alapú mappaszerkezetben, a backend pedig controller, service, repository és DTO rétegekre bontva készült.
4. Bővíthetőség: új játékforrástípusok, új admin funkciók és új ranglista-szűrések később hozzáadhatók.
5. Adatvalidáció: a DTO-k validációs annotációkat használnak, a képfeltöltés csak megengedett képtípusokat fogad el.
6. Lokalizáció: a backend és frontend is támogat magyar és angol szövegeket.

4. Technológiák és fejlesztőeszközök

4.1 Backend

- Java 17
- Spring Boot 4.0.1
- Spring Web / Spring Security / Spring Data JPA
- MySQL Connector/J / Maven / JWT

4.2 Frontend

- React 19
- React Router DOM / Vite
- CSS moduláris stílusfájlok / Fetch API

4.3 Adatbázis

- MySQL
- JPA/Hibernate ORM

4.4 Fejlesztőeszközök

- IntelliJ IDEA
- Node.js és npm
- Git verziókezelés
- Böngésző fejlesztői eszközök
- Maven Wrapper

5. Fejlesztési módszertan

A projekt fejlesztése iteratív módon történt. A funkciók kisebb egységekre bontva készültek el. A projekt egyéni munkaként került beadásra.

5.1 Feladatkövetés

A feladatok backlog jelleggel kezelhetők. A főbb epicek:

- Felhasználókezelés
- Játékkezelés
- Ranglista és pontmentés
- Admin felület
- Többnyelvűség
- Tesztelés és dokumentáció

5.2 Verziókezelés

A projekt Git verziókezelővel kezelhető. A közös projektgyökér tartalmazza a backend és frontend mappákat:

- `FinalProject` - Spring Boot backend
- `gameverse-frontend` - React frontend

5.3 Szerepek

A projekt egyéni munkaként került beadásra, és a következő részeket látta el:

- backend fejlesztő,
- frontend fejlesztő,
- adatbázis-tervező,
- tesztelő,
- dokumentáció készítő.

5.4 Minőségbiztosítási terv

- Kód fordítása backend és frontend oldalon.
- Backend unit tesztek futtatása.
- Frontend build ellenőrzése.
- Manuális UI tesztelés böngészőben.
- Edge case-ek ellenőrzése: hibás login, üres mezők, jogosultság nélküli admin oldalak, nem támogatott képfájl feltöltése.

6. Rendszeráttekintés és architektúra

6.1 Magas szintű architektúra

A rendszer három fő komponensből áll:

Komponens	Leírás
React frontend	Felhasználói felület, oldalak, űrlapok, REST hívások
Spring Boot backend	REST API, üzleti logika, autentikáció, fájlfeltöltés
MySQL adatbázis	Felhasználók, játékok, pontok, értékelések tárolása

Egyszerűsített működés:

Felhasználó böngészője → React + Vite frontend →

Spring Boot REST API → MySQL adatbázis

6.2 Backend rétegek

A rendszer szerveroldali logikája egy jól elkülönített, többretegű architektúrára épül, amely biztosítja a kód átláthatóságát és a feladatkörök tiszta szétválasztását az alábbiak szerint:

- Controller réteg: HTTP végpontok kezelése (`AuthController`, `GameController`, `AdminGameController`, `ScoreController`, `RatingController`, `LeaderboardController`).
- Service / support réteg: üzleti logika, például pontmentés, képfeltöltés, inline HTML bridge, profil törlés.
- Repository réteg: JPA repository-k az adatbázis-műveletekhez.
- DTO réteg: request és response objektumok.
- Security réteg: JWT token feldolgozás, admin/user jogosultságok, `@AuthUser` resolver.

6.3 Frontend szerkezet

A frontend feature-alapú szerkezetet használ:

- `features/auth` - login, register, jelszó-visszaállítás
- `features/home` - kezdőlap, játéklista
- `features/game` - játékkoldal, játékterület, játékon belüli ranglista
- `features/leaderboard` - globális ranglista
- `features/profile` - profilkezelés
- `features/admin` - admin felület
- `lib/api` - API kliensek
- `i18n` - fordítások és nyelvkezelés

6.4 Jogosultsági szintek

A rendszer biztonságos működését és az erőforrásokhoz való hozzáférést egy differenciált szerepkörkezelő rendszer szabályozza, amely az alábbi jogosultsági szinteket különíti el:

Szerep	Jogosultság
Vendég	Játékok megtekintése, ranglista megtekintése, regisztráció, belépés
Felhasználó	Profilkezelés, értékelés, pontmentés, profil törlés
Admin	Felhasználók kezelése, játékok kezelése, játék előnézet és publikálás

7. Adatbázis dokumentáció

Az alkalmazás adatszerkezetét úgy terveztük meg, hogy hatékonyan kiszolgálja a felhasználói interakciókat, a játékmenetek naplózását és a biztonságos hitelesítést.

7.1 Táblák

Az adatok strukturált tárolásáért az alábbi fő entitások felelősek, amelyek elkülönítve kezelik a felhasználói, tartalmi és eredményességi adatokat:

Tábla	Feladat
users	felhasználói adatok, e-mail, jelszó hash, profilkép
games	játékok adatai, többnyelvű cím/leírás, forrástípus
scores	pontszámok felhasználóhoz és játékhoz kapcsolva

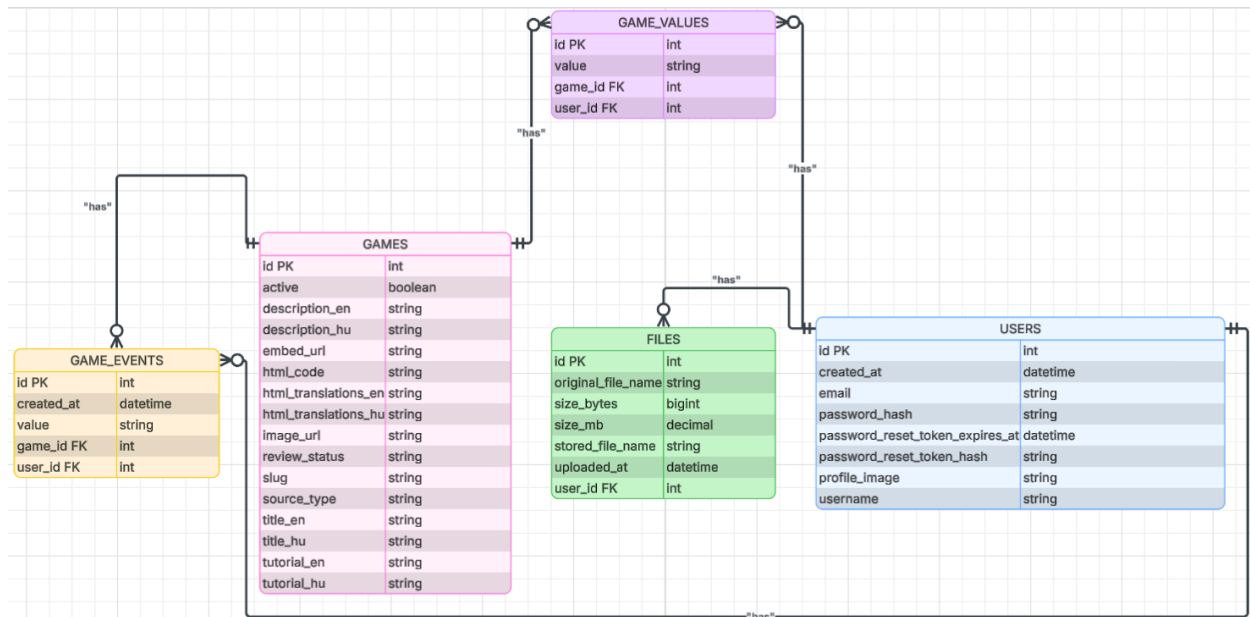
7.2 Kapcsolatok

A rendszer integritását a táblák közötti logikai összeköttetések biztosítják, amelyek lehetővé teszik az adatok közötti keresztlekérdezéseket:

- Egy felhasználónak több pontszáma lehet.
- Egy játéknak több pontszáma lehet.
- Egy felhasználó több játékot értékelhet.
- Egy játék több értékelést kaphat.
- Egy felhasználóhoz több profilkép-feltöltési rekord tartozhat.

7.3 ER diagram

Az adatbázis vizuális modellje szemlélteti az entitások hierarchiáját és a közöttük fennálló idegen kulcs (Foreign Key) kapcsolatokat:



7.4 Normalizálás

Az adatbázis szerkezete törekszik a 3NF elveire:

- a felhasználói adatok külön `users` táblában vannak,
- a játékadatok külön `games` táblában vannak,
- a pontszám kapcsolótábla jelleggel kapcsolja a felhasználókat és játékokat,
- az értékelések külön táblában vannak, egyedi user-game kapcsolat alapján,
- a feltöltési napló külön táblában tárolódik.

7.5 Indexek és megszorítások

A lekérdezések gyorsítása és az adatintegritás fenntartása érdekében az alábbi egyedi kényszereket és indexelési stratégiákat alkalmazzuk:

- `users.email` egyedi.
- `games.slug` egyedi.
- `game\_ratings` táblában a user-game pár egyedi.
- A pontszámoknál index van a `game\_id` és `user\_id` mezőkön.
- A profilkép feltöltéseknél index van a `user\_id` mezőn.

8. API dokumentáció

Az API REST alapú, JSON formátumot használ az adatcseréhez. A biztonsági réteg JWT tokeneket használ a felhasználók azonosítására.

8.1 Auth API

Metódus	Végpont	Leírás	Jogosultság
POST	`/api/auth/register`	új felhasználó regisztrációja	publikus
POST	`/api/auth/login`	belépés, JWT token visszaadása	publikus
GET	`/api/auth/me`	aktuális felhasználó lekérése	user/admin

Metódus	Végpont	Leírás	Jogosultság
PUT	<code>`/api/auth/me`</code>	saját profil módosítása	user
DELETE	<code>`/api/auth/me`</code>	saját profil törlése	user
POST	<code>`/api/auth/me/profile-image`</code>	profilkép feltöltése	user
POST	<code>`/api/auth/request-password-reset`</code>	jelszó-visszaállítási token kérése	publikus
POST	<code>`/api/auth/reset-password`</code>	jelszó visszaállítása	publikus

Példa login requestre:

```
{
  "email": "player@example.com",
  "password": "secret123"
}
```

Példa login response:

```
{
  "userId": 1,
  "username": "Player",
```

```
"email": "player@example.com",
"role": "USER",
"token": "jwt-token"
}
```

8.2 Game API

A játékok kezeléséért felelős interfész biztosítja a publikus böngészést, a játékmenetek indítását, valamint a regisztrált felhasználók számára az interaktív funkciók elérését az alábbi végpontokon keresztül:

Metódus	Végpont	Leírás	Jogosultság
GET	<code>/api/games`</code>	aktív játékok listája	publikus
GET	<code>/api/games/{slug}`</code>	játék részletei slug alapján	publikus
GET	<code>/api/games/{slug}/play`</code>	inline HTML játék futtatása (bridge)	publikus
POST	<code>/api/games/{gameId}/like`</code>	játék like	user
POST	<code>/api/games/{gameId}/dislike`</code>	játék dislike	user

8.3 Score és leaderboard API

Metódus	Végpont	Leírás	Jogosultság
POST	<code>/api/scores`</code>	pontszám mentése	user
GET	<code>/api/scores/leaderboard/{gameId}`</code>	top lista játékonként	publikus
GET	<code>/api/leaderboard`</code>	globális ranglista	publikus
GET	<code>/api/leaderboard/{gameId}`</code>	játék szerinti ranglista	publikus

Példa pontmentés:

```
{  
  "gameId": 1,  
  "score": 140  
}
```

8.4 Admin user API

Ez a modul teszi lehetővé a rendszergazdák számára a felhasználói adatbázis teljes körű menedzselését, beleértve a fiókok létrehozását, módosítását és a hozzáférések szabályozását:

Metódus	Végpont	Leírás	Jogosultság
GET	`/api/admin/users`	felhasználók listázása	admin
POST	`/api/admin/users`	felhasználó létrehozása	admin
PUT	`/api/admin/users/{userId}`	felhasználó módosítása	admin
DELETE	`/api/admin/users/{userId}`	felhasználó törlése	admin

8.5 Admin game API

A játékkezelő felület az adminisztrátorok számára biztosít exkluzív eszközöket a tartalomkezeléshez, a feltöltési folyamatokhoz és a játékok életciklusának (publikálás, módosítás) felügyeletéhez:

Metódus	Végpont	Leírás	Jogosultság
GET	`/api/admin/games`	összes játék listázása	admin
GET	`/api/admin/games/{gameId}`	játék admin részletei	admin
POST	`/api/admin/games`	játék létrehozása	admin
PUT	`/api/admin/games/{gameId}`	játék módosítása	admin
DELETE	`/api/admin/games/{gameId}`	játék törlése	admin
PATCH	`/api/admin/games/{gameId}/status`	publikálási státusz módosítása	admin
GET	`/api/admin/games/{slug}/preview`	játék admin előnézete	admin
POST	`/api/admin/games/image`	játékkép feltöltése	admin

8.6 Hibakezelés

A rendszer stabilitását és a felhasználói élményt egy egységesített hibakezelési mechanizmus támogatja, amely biztosítja, hogy a váratlan eseményekről a kliensoldal mindig strukturált értesítést kapjon.

A backend hibák jellemzően az alábbi JSON formátumban térnek vissza, megkönnyítve az automatizált feldolgozást:

```
{  
  "error": "Hibaüzenet"  
}
```

A fejlesztés során az alábbi gyakori hibaforrásokat különítettük el, amelyeket a rendszer célzottan kezel:

- Validációs hibák (400 Bad Request): Hibás formátumú e-mail címek, túl rövid jelszavak vagy kötelezően kitöltendő mezők hiánya esetén lépnek fel.
- Hitelesítési és jogosultsági hibák (401 Unauthorized / 403 Forbidden): Lejárt JWT tokenek, érvénytelen bejelentkezési adatok, vagy ha egy egyszerű felhasználó adminisztrátori végpontot próbál elérni.
- Erőforrás hibák (404 Not Found): Amikor a kért játék, pontszám vagy felhasználói profil nem található az adatbázisban.
- Üzleti logika ütközések (409 Conflict): Például már foglalt e-mail címmel történő regisztrációs kísérlet vagy duplikált értékelés leadása.
- Szerveroldali hibák (500 Internal Server Error): Váratlan adatbázis-leállás vagy egyéb, nem várt futásidejű kivételek kezelése.

A frontend az errors.js fájlban tárolt szótár segítségével fordított, felhasználóbarát hibaüzeneteket rendel a kapott válaszokhoz.

9. Telepítési és futtatási útmutató

Ez a fejezet lépésről lépésre bemutatja, hogyan állítható fel a fejlesztői környezet, és miként indítható el az alkalmazás mind a szerver, mind a kliens oldalon.

9.1 Előfeltételek

- Java 17 vagy újabb kompatibilis JDK
- Maven Wrapper a backend mappában
- Node.js és npm
- MySQL szerver
- Git

9.2 Adatbázis beállítása

Az alkalmazás alapértelmezett adatbázis-beállítása:

```
spring.datasource.url=jdbc:mysql://localhost:3306/Gameverse?createDatabaseIfNotExist=true&useSSL=false&zeroDateTimeBehavior=convertToNull&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
```

Ezek a beállítások az `application.properties` fájlban találhatók, és a Spring DataSource (adatforrás) konfigurálásáért felelnek.

1. `spring.datasource.url`

- a. Ez a legfontosabb sor, amely meghatározza az adatbázis helyét és a kapcsolódás módját.
- b. `jdbc:mysql://localhost:3306/Gameverse:` Megadja, hogy a MySQL protokollon keresztül a saját gépünkön (localhost), a 3306-os alapértelmezett porton keressük a Gameverse nevű adatbázist.
- c. `createDatabaseIfNotExist=true:` Ha még nincs Gameverse nevű adatbázis a szerveren, az alkalmazás induláskor automatikusan létrehozza azt.
- d. `useSSL=false:` Kikapcsolja az SSL-titkosítást. Fejlesztői környezetben ez gyakori a bonyolult tanúsítványkezelés elkerülése végett.
- e. `allowPublicKeyRetrieval=true:` Lehetővé teszi a nyilvános kulcs lekérését a szervertől, ami bizonyos MySQL hitelesítési módoknál (pl. `caching_sha2_password`) szükséges a sikeres kapcsolódáshoz.

2. `spring.datasource.username` és `password`

- a. `username=root:` Az adatbázis-felhasználó neve. A root a MySQL rendszergazdai fiókja.
- b. `password=:` A felhasználóhoz tartozó jelszó. Ebben az esetben üresen van hagyva (ami gyakori alapértelmezett beállítás a lokális XAMPP telepítéseknél).

3. `spring.jpa.hibernate.ddl-auto=update`

- a. Ez a beállítás vezérli, hogy a Hibernate (az adatbázis-kezelő keretrendszer) mit kezdjen a táblákkal az alkalmazás indításakor:
- b. `update:` A leggyakrabban használt mód fejlesztés alatt. Összehasonlítja a Java entitásokat az adatbázis tábláival. Ha új mezőt adsz a kódhoz, azt automatikusan létrehozza az adatbázisban is, a meglévő adatokat nem törli le.

9.3 Backend futtatása

A szerveroldali alkalmazás indításához és a forráskód fordításához a beépített Maven segédprogramot használhatjuk az alábbi parancsok kiadásával:

```
cd /FinalProject/FinalProject  
./mvnw spring-boot:run
```

Backend fordítás:

```
./mvnw -DskipTests compile
```

9.4 Frontend futtatása

A felhasználói felület kiszolgálásához először a függőségek telepítése, majd a fejlesztői szerver elindítása szükséges a kliens könyvtárából:

```
cd /FinalProject/gameverse-frontend  
npm install  
npm run dev
```

Frontend build:

```
npm run build
```

9.5 Admin belépés

A rendszer első használatba vételekor a felügyeleti funkciók eléréséhez az alábbi előre definiált hitelesítési adatokkal lehet bejelentkezni:

E-mail: admin@admin.com

Jelszó: admin

Ezek az `application.properties` fájlban módosíthatók.

10. Használati útmutató

A rendszer három fő felhasználói szintet különít el: vendég, regisztrált felhasználó és adminisztrátor.

10.1 Vendég felhasználó

A rendszer korlátozott, de széles körű hozzáférést biztosít az azonosítatlan látogatók számára, lehetővé téve a platform tartalmának felfedezését és az alapvető információszerzést az alábbi interakciókon keresztül:

- megnyithatja a kezdőlapot,
- böngészheti és keresheti a játékokat,
- megnyithatja a játéköldalakat,
- megnézheti a ranglistát,
- regisztrálhat vagy bejelentkezhet.

10.2 Regisztráció és bejelentkezés

A regisztrációs oldalon a felhasználó megadja:

- felhasználónév,
- e-mail cím,
- jelszó.

Regisztráció

Felhasználónév
Hogyan szólíthatunk?

E-mail cím
example@email.com

Jelszó
Legalább 6 karakter

Fiók létrehozása

Van már fiókod? [Bejelentkezés!](#)

Belépés után a rendszer JWT tokenet tárol a kliensoldalon, és a védett kéréseknél ezt küldi a backendnek.

10.3 Profiloldal

A profiloldalon a felhasználó:

- módosíthatja felhasználónevét,
- módosíthatja e-mail címét,
- új jelszót állíthat be,
- profilképet tölthet fel,
- nyelvet válthat magyar és angol között,
- kijelentkezhet,
- megerősítés után törölheti saját profilját.



Profilom

Itt frissítheted a fiókadataidat.

Felhasználónév

E-mail

Új jelszó

Profilkép feltöltése

 Nincs fájl kiválasztva

Engedélyezett: JPG, PNG, WEBP, GIF

Nyelv
Válaszd ki, milyen nyelven jelenjen meg az oldal.

Oldal nyelve

Magyar

[Mentés](#) [Kijelentkezés](#) [Profil törlése](#)

A(z) localhost:5173 közléndője

Biztosan törölni szeretnéd a profilodat? Ez a művelet nem vonható vissza.

[OK](#) [Mégse](#)



Profilom

Itt frissítheted a fiókadataidat.

Felhasználónév

E-mail

Új jelszó

Profilkép feltöltése

 Nincs fájl kiválasztva

Engedélyezett: JPG, PNG, WEBP, GIF

Nyelv
Válaszd ki, milyen nyelven jelenjen meg az oldal.

Oldal nyelve

Magyar

[Mentés](#) [Kijelentkezés](#) [Profil törlése](#)

10.4 Kezdőlap

A kezdőlapon megjelennek az aktív játékok. A keresőmező a játék címében és leírásában keres. A legtöbb like-kal rendelkező játékok külön kiemelt részen jelenhetnek meg.

10.5 Játékoldal

A játékoldal három fő részből áll:

- bal oldali játékösszefoglaló,
- középső játékterület,
- jobb oldali játékspecifikus ranglista.

Inline HTML játék esetén a rendszer jelzi, hogy a pontokat automatikusan menti. Embedded játék esetén jelzi, hogy a pontok mentése nem lehetséges, ha a külső játék nem támogatja a pontküldést.

10.6 Globális ranglista

A globális ranglista oldalon:

- játék szerint lehet szűrni,
- konkrét dátum alapján lehet szűrni,
- pontszám szerint növekvő vagy csökkenő sorrend választható,
- a szűrők törölhetők.

10.7 Admin felület

Az adminisztrátor:

- felhasználókat listázhat, létrehozhat, módosíthat és törölhet,
- játékokat listázhat, létrehozhat, módosíthat és törölhet,
- játékot előnézetben megtekinthet,
- játékképet tölthet fel,
- publikálási státuszt állíthat.

A játék hozzáadásakor a slug automatikusan generálódik a játék címéből. Ha a magyar és angol cím eltér, a slug az angol cím alapján készül.

11. Tesztelési dokumentáció

11.1 Tesztelési célok

A tesztelés célja annak ellenőrzése, hogy:

- a backend üzleti logikája megfelelően működik,
- a REST végpontok megfelelő adatokat adnak vissza,
- a jogosultsági mechanizmus működik,
- a fájlfeltöltés validálja a képtípusokat,
- a frontend build hibamentesen elkészül,
- a fő felhasználói folyamatok manuálisan használhatók.

11.2 Automatikus backend tesztek

A projektben több JUnit teszt található, például:

Tesztfájl	Ellenőrzött terület
`AuthControllerTest`	login, jelszó-visszaállítás
`JwtServiceTest`	JWT generálás és validálás
`JwtAuthFilterTest`	auth filter működés
`AuthUserResolverTest`	aktuális user azonosítása
`AdminControllerTest`	admin felhasználókezelés
`ScoreServiceTest`	pontmentési logika
`ScoreControllerTest`	score végpont
`RatingControllerTest`	like/dislike logika
`GameImageStorageServiceTest`	játékkép validálás

Tesztfájl	Ellenőrzött terület
`ProfileImageStorageServiceTest`	profilkép validálás
`LocalizationServiceTest`	fordítások betöltése

11.3 Futtatott ellenőrzések

A dokumentáció készítésekor az alábbi ellenőrzések sikeresen lefutottak:

```
cd /FinalProject/gameverse-frontend
npm run build
```

Eredmény: sikeres frontend production build.

```
cd /FinalProject/FinalProject
./mvnw -DskipTests compile
```

Eredmény: sikeres backend fordítás.

11.4 Megjegyzés a tesztkörnyezetről

A projektben léteznek unit tesztek, viszont a jelenlegi helyi Java 21 környezetben néhány Mockito-alapú teszt futtatásakor Byte Buddy agent attach hiba jelentkezhethet. Ez környezeti futtatási probléma, nem közvetlen logikai hiba.

11.5 Manuális tesztesetek

Teszteset	Lépések	Elvárt eredmény
Regisztráció	Új felhasználó létrehozása érvényes adatokkal	Sikeres regisztráció és bejelentkezés
Hibás login	Hibás e-mail vagy jelszó megadása	Fordított hibaüzenet jelenik meg
Profil módosítás	Név vagy e-mail módosítása	Profiladatok frissülnek
Profil törlés	Profil törlése gomb, megerősítés	Fiók törlődik, felhasználó kijelentkezik
Játék keresése	Kezdőlapon keresőmező használata	Csak illeszkedő játékok látszanak
Pontmentés	Inline játék pontot küld	Pontszám bekerül a ranglistába
Ranglista dátumszűrés	Dátum kiválasztása	Csak az adott napi eredmények jelennek meg

Teszteset	Lépések	Elvárt eredmény
Like/dislike	Bejelentkezett user értékel	Számlálók frissülnek
Admin játék létrehozás	Admin új játékot ad hozzá	Játék bekerül az adatbázisba
Jogosultság nélküli admin oldal	Normál user `/admin` útvonalat nyit	Visszairányítás történik

12. Kódminőség és biztonsági megoldások

12.1 Tiszta kód

A projektben a kód több fájlra és felelősségi körre van bontva. A backend controller, DTO, service/support és repository rétegeket használ. A frontend feature-alapú szerkezete segíti az áttekinthetőséget.

12.2 Biztonság

Az alkalmazás adatainak és integritásának védelmét egy többszintű biztonsági architektúra garantálja, amely a modern iparági szabványokra és a szigorú hozzáférés-szabályozásra épül:

- JWT token alapú autentikáció.
- Admin végpontok ``ROLE_ADMIN`` jogosultsághoz kötve.
- Felhasználói műveletek bejelentkezéshez kötve.
- Jelszavak BCrypt hasheléssel tárolódnak.
- Képfeltöltésnél csak JPG, PNG, WEBP és GIF engedélyezett.
- Profil törlése előtt a frontend megerősítést kér.
- Admin felületre normál felhasználó nem léphet be.

12.3 Lokalizáció

A frontend fordításai a ``translations.js`` fájlban vannak. A backend a ``messages.properties`` és ``messages_hu.properties`` fájlokat használja. A nyelvválasztás a profiloldalon állítható.

13. Továbbfejlesztési lehetőségek

13.1 Funkcionális bővítések

A felhasználói élmény fokozása és a közösségi interakciók mélyítése érdekében a projekt hosszú távú céljai között az alábbi funkcionális fejlesztések szerepelnek:

- Teljes achievement rendszer.
- Barátlista és privát kihívások.
- Részletes felhasználói statisztikák.
- Kommentelési lehetőség játékok alatt.
- Moderátori szerepkör.
- Admin audit napló.
- Több pontszám-kategória játékonként.

13.2 Nem funkcionális bővítések

A szoftver minőségének, skálázhatóságának és az üzemeltetés hatékonyságának növelése érdekében a technológiai háttér az alábbi irányokba fejleszthető tovább:

- CI/CD pipeline GitHub Actions segítségével.
- Docker alapú futtatás.
- Automatikus adatbázis migráció Flyway vagy Liquibase segítségével.
- Nagyobb automatikus tesztlefedettség.
- E2E tesztelés Playwright vagy Cypress segítségével.
- Objektumtároló használata képekhez.
- Production környezethez erősebb titokkezelés.

14. Források

-
- Spring Boot dokumentáció: <https://spring.io/projects/spring-boot>
 - React dokumentáció: <https://react.dev>
 - Vite dokumentáció: <https://vite.dev>
 - MySQL dokumentáció: <https://dev.mysql.com/doc>
 - Képzési és kimeneti követelmények:
`Szoftverfejlesztő-és-tesztelő-2023.11.21.-v3.pdf`
 - Final Project requirements: `Final Project requirements - Software developer and -testers (Java).pdf`
 - Projekt forráskód: `FinalProject`